

Tarea - Creación y documentación de un componente personalizado (Swing)	1
Descripción del componente	1
Clase Base Utilizada	1
Problema que resuelve:	1
Propiedades añadidas:	1
Eventos Soportados:	1
Como se interactúa:	1
Código:	2
Clase MiComponenteX:	2
Clase Hilo:	4

Tarea - Creación y documentación de un componente personalizado (Swing)

Descripción del componente:

Un JButton que al pasar el ratón por “encima” del boton este cambia de color de fondo al elegido por el usuario, si mantiene el botón el número de segundos establecido por el usuario, este cambiar de color a otro (también elegido por el usuario). Esto solo ocurrirá si no se producen un número x de clicks que entonces el botón se desactivara cambiando de color y desactivando tambien las interacciones con este.

Clase Base Utilizada: JButton

Problema que resuelve:

Cambio de color de fondo al hacer hover sobre un botón o al estar desactivado luego de un número de clicks.

Propiedades añadidas:

Color de fondo

Número máximo de clicks

Segundos “hover” para fondo alternativo

Eventos Soportados:

Hover (MouseEntered/MouseExited)

Click (ActionPerformed)

Como se interactúa:

Pasando el ratón por encima y/o clickando en él

Código:

Clase MiComponenteX:

```
class MiComponenteX extends JButton
{
    private Color colorFondo;
    private Color colorHover;
    private Color colorDesactivado;
    private int maxClicks;
    private int numClicks;
    private Color colorHoverAlt;
    private Hilo hilo;
    private int segundosHoverAlt;

    public MiComponenteX() {
        super();
        maxClicks=5;
        numClicks=0;
        colorFondo=new Color(0,0,0);
        colorHover=new Color(255,255,255);
        colorDesactivado=new Color(255,0,255);
        colorHoverAlt=new Color(36,64,45);
        segundosHoverAlt=5;
        this.setBackground(colorFondo);
        MiComponenteX boton=this;

        this.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                if (boton.isEnabled())
                {
                    hilo=new Hilo(boton);
                    hilo.start();
                    boton.setBackground(colorHover);
                }
            }
            @Override
            public void mouseExited(MouseEvent e) {
                if (boton.isEnabled())
                {
                    hilo.interrupt();
                    boton.setBackground(colorFondo);
                }
            }
        });
    }
}
```

```

        boton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                numClicks++;
                if (numClicks==maxClicks)
                {
                    boton.setEnabled(false);
                    boton.setBackground(colorDesactivado);
                }
            }
        });
    }

    public Color getColorFondo() {
        return colorFondo;
    }

    public void setColorFondo(Color colorFondo) {
        this.colorFondo = colorFondo;
    }

    public void setColorFondo(int r,int g,int b) {
        this.colorFondo = new Color(r,g,b);
    }

    public Color getColorHover() {
        return colorHover;
    }

    public void setColorHover(Color colorHover) {
        this.colorHover = colorHover;
    }

    public void setColorHover(int r,int g,int b) {
        this.colorHover = new Color(r,g,b);
    }

    public Color getColorDesactivado() {
        return colorDesactivado;
    }

    public void setColorDesactivado(Color colorDesactivado) {
        this.colorDesactivado = colorDesactivado;
    }

    public void setColorDesactivado(int r,int g,int b) {
        this.colorDesactivado = new Color(r,g,b);
    }

```

```

public int getMaxClicks() {
    return maxClicks;
}

public void setMaxClicks(int maxClicks) {
    this.maxClicks = maxClicks;
}

public int getNumClicks() {
    return numClicks;
}

public void reiniciarClicks()
{
    this.numClicks=0;
    this.setEnabled(true);
}

public Color getColorHoverAlt() {
    return colorHoverAlt;
}

public void setColorHoverAlt(Color colorHoverAlt) {
    this.colorHoverAlt = colorHoverAlt;
}

public int getSegundosHoverAlt() {
    return segundosHoverAlt;
}

public void setSegundosHoverAlt(int segundosHoverAlt) {
    this.segundosHoverAlt = segundosHoverAlt;
}
}

```

Clase Hilo:

```

class Hilo extends Thread
{
    private MiComponenteX boton;
    private int segundos;

    public Hilo(MiComponenteX boton) {
        this.boton = boton;
    }
}

```

```
@Override
public void run() {
    for (int i=0;i<boton.getSegundosHoverAlt();i++)
    {
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            break;
        }
        if (i==boton.getSegundosHoverAlt()-1)
        {
            boton.setBackground(boton.getColorHoverAlt());
        }
    }
}
}
```

